



PasswordStore Audit Report

Project

PasswordStore

Repository

<https://github.com/molalign8468/30-day-security-sprint.git>

Commit Hash

[613f5d4f6064dd488b5cbdb23d4bafdc38501811](#)



AUGUST 1, 2026

MOLALIGN GETAHUN

Table of Content

Table of Content 1

Protocol Summary..... 2

Roles 2

Executive Summary 2

Issues found 2

Scope & Methodology..... 2

Scope 2

Methodology:..... 3

Risk Classification 3

Severity Rating..... 3

Findings Summary 3

Detailed Findings 4

[CRIT-01] Passwords stored on-chain are visible to anyone, not matter solidity variable visibility..... 4

[CRIT-02] PasswordStore::setPassword is callable by anyone 5

Disclaimer 6

Conclusion 6

Protocol Summary

PasswordStore is a protocol dedicated to storage and retrieval of a user's passwords. The protocol is designed to be used by a single user, and is not designed to be used by multiple users. Only the owner should be able to set and access this password.

Roles

- Owner: Is the only one who should be able to set and access the password.

For this contract, only the owner should be able to interact with the contract.

Executive Summary

Status: CRITICAL SEVERITY

The assessment of the **PasswordStore** protocol identified **two security findings**, which are **Critical** severity vulnerabilities. These issues undermine the protocol's intended security model by exposing sensitive data and allowing unauthorized interaction with owner-restricted functionality. Addressing these findings before deployment is strongly recommended to preserve the protocol's confidentiality and access control guarantees.

Issues found

Severity	Count	Status
Critical	2	Unresolved
High	0	-
Medium	0	-
Low	0	-
Informational	0	-

Scope & Methodology

Scope

File	Type	Logic
Day5_Password_Store.sol	Smart Contract	setPassword() and getPassword ()

Methodology:

- Manual code review
- Threat modeling
- Foundry-based exploit testing

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High / Medium	Medium
	Medium	High / Medium	Medium	Medium / Low
	Low	Medium	Medium / Low	Low

Severity Rating

Critical	Full fund loss or protocol takeover
High	Significant fund risk
Medium	Logic or trust issues
Low	Minor risk
Informational	Best practices

Findings Summary

CRIT-01	CRITICAL	Passwords stored on-chain are visible to anyone, not matter solidity variable visibility
CRIT-02	CRITICAL	PasswordStore::setPassword is callable by anyone

Detailed Findings

[CRIT-01] Passwords stored on-chain are visible to anyone, not matter solidity variable visibility

Severity: Critical

Location: Day5_Password_Store.sol:L14

Description: All data stored on-chain is visible to anyone, and can be read directly from the blockchain.

The PasswordStore::s_password variable is intended to be a private variable, and only accessed through the PasswordStore::getPassword function, which is intended to be only called by the owner of the contract.

However, anyone can directly read this using any number of off chain methodologies

Proof of Concept: The below test case shows how anyone could read the password directly from the blockchain. We use foundry's cast tool to read directly from the storage of the contract, without being the owner.

Create a locally running chain

```
1 make anvil
```

1. Deploy the contract to the chain

```
1 make deploy
```

2. Run the storage tool

We use 1 because that's the storage slot of s_password in the contract.

```
1 cast storage <ADDRESS_HERE> 1 --rpc-url http://127.0.0.1:8545
```

You'll get an output that looks like this:

```
0x6d7950617373776f726400000000000000000000000000000000000000000014
```

You can then parse that hex to a string with:

```
1 cast parse-bytes32-string
```

```
0x6d7950617373776f726400000000000000000000000000000000000000000014
```

And get an output of:

```
1 myPassword
```

Risk Evaluation:

- **Impact:** High. An attacker can retrieve the plaintext password and bypass the intended access control.
- **Likelihood:** High. The password can be read by anyone using publicly available blockchain/RPC tools without privileged access.
- **Overall Severity:** Critical.

Recommended Mitigation: Due to this, the overall architecture of the contract should be rethought. One could encrypt the password off-chain, and then store the encrypted password on-chain. This would require the user to remember another password on-chain to decrypt the password. However, you'd also likely want to remove the view function as you wouldn't want the user to accidentally send a transaction with the password that decrypts your password.

[CRIT-02] PasswordStore::setPassword is callable by anyone

Severity: Critical

Location: Day5_Password_Store.sol:L26-29

Description: The `PasswordStore::setPassword` function is set to be an `external` function, however then aspect of the function and over all purpose of the smart contract is that `This function allows only the owner to set a new password`.

```
1     function setPassword(string memory newPassword) external {
2  @>    // @audit - There are no access controls here
3         s_password = newPassword;
4         emit SetNetPassword();
5     }
```

Proof of Concept:

Add the following to the `PasswordStore.t.sol` test suite.

```
1     function test_anyone_can_set_password(address randomAddress) public {
2         vm.prank(randomAddress);
3         string memory expectedPassword = "myNewPassword";
4         passwordStore.setPassword(expectedPassword);
5         vm.prank(owner);
6         string memory actualPassword = passwordStore.getPassword();
7         assertEq(actualPassword, expectedPassword);
8     }
```

Risk Evaluation:

- **Impact:** High. Anyone can set/change the password of the contract.
- **Likelihood:** High. The function is publicly callable and requires no privileged access.
- **Overall Severity:** Critical.

Recommended Mitigation: Add an access control modifier to the `setPassword` function.

```
1 modifier onlyOwner() {  
  require(msg.sender == s_owner, PasswordStore__NotOwner());  
  _;  
}  
  
function setPassword(string memory newPassword) external onlyOwner {  
  s_password = newPassword;  
  emit SetNewPassword();  
}
```

Disclaimer

I, Molalign Getahun, make every effort to identify as many vulnerabilities as possible within the allotted review period. However, I assume no responsibility or liability for the findings presented in this document. This security audit does not constitute an endorsement of the underlying business or product. The audit was timeboxed, and the review was limited solely to the security aspects of the Solidity implementation of the reviewed contracts.

Conclusion

Deployment should not proceed until the issue is fixed and the contract is re-audited.